

Introduction:

Le sujet est basé sur un célèbre jeu que l'on connaît tous Flappy Bird , c'est très simple il s'agit d'un jeu où un oiseau doit chevaucher des tuyaux sans les toucher, et ni même toucher le sol ainsi que le plafond .L'objectif principal de ce projet à travers le jeu de Flappy Bird est d'entraîner un agent autrement dit un programme via l'algorithme de NEAT. Comme nous pouvons le voir dans le Glossaire NEAT est un algorithme génétique qui fait évoluer les points de connexion ainsi que la structure du réseau de neurones , en y ajoutant des nœuds et des corrections au fil des générations.

Étape 1) Bot naïf : règles codées en dur

Le jeu à vue d'œil est très simple: la gravité tire l'oiseau vers le bas et chaque clique ou activation sur la touche espace le projette vers le haut.

Lors de mes tests manuels, je me suis rendu compte que ce qui semblait être un jeu facile et naïf était en fait un jeu assez compliqué dans le fond car à peine la partie activée l'oiseau se retrouvait déjà au sol également quand je tentais de lui faire survoler les tuyaux ,soit il se les prenait rapidement , soit il touchait rapidement le sol ou le plafond en, il y avait donc très peu de chance de gagner d'ailleurs sur 5 parties, je n'en ai gagnée aucune.

Alors je me suis donc intéressé au fichier de configuration et je me suis rendu compte que les paramètres étaient assez étrange, une gravité faible de 0.5 Une vitesse de saut en négatif, une vitesse maximum de 12.0 et donc le palier de difficulté qui ne serait pas trop grand? , et d'ailleurs comment se fait-il que les tuyaux soient plus gros que le sol par exemple. Ainsi, si un jour l'on veut espérer pouvoir gagner au moins une fois à ce jeu, il faut donc optimiser ses paramètres.

Étape 2) Premier réseau de neurones : perceptron sans entraînement

Et donc pour optimiser ses paramètres, le réseau de neurones va devoir apprendre en prenant compte de plusieurs signaux .Avant de procéder à tout ça pour vraiment se rendre compte des données et du fonctionnement des jeux, ainsi que de ses paramètres, j'ai fais dans un premier temps comme conseiller des modifications manuelles donc c'est-à-dire que on est passé d'un np.random a des valeurs numériques choisies. Les valeurs numériques à modifier étaient les state() entre parenthèses ce qui correspond aux vecteurs d'État, comme nous avons pu le voir et dans notre jeu, nous avons cinq vecteurs d'État ce qui signifie cinq données numériques à paramétrer.

ces vecteurs numériques sont normalisé entre -1 et un ou entre zéro et un et selon la valeur on va trier les vecteurs via leur index :

-State[0] = On peut le mettre à 0.0 car il s'agit de la position verticale qui n'est pas absolue car ce qui compte c'est la position par rapport au gap, c'est-à-dire entre parenthèses l'espace vide.

-State[1] = Vitesse, on passe en négatif parce que en position positif, l'oiseau monte très vite et donc il serait peut-être logique de le faire basculer en négatif et pour rester sur une échelle raisonnable on va mettre à -1.0

-State[2] = C'est la distance horizontale au prochain tuyau, on peut la mettre également à 0.0, car c'est la moins critique.

-State[3] = C'est la différence entre la position de l'oiseau et le bord du haut du gap, la cette notion c'est vraiment le cœur du TP, vu que si l'oiseau est dans le gap c'est quelque chose de positif donc on va forcément le mettre en valeur positive et on va également le mettre en un 1.0

-State[4] = Différence du port bas du gap, donc quelque chose qui est aussi utile, on va le mettre en positif en 1.0

J'ai essayé d'ajuster selon ma logique et celle des vecteurs, mais j'ai remarqué qu'en faisant une manipulation manuelle et donc en enlevant des chiffres random, je tombais quand même sur un score à zéro. Dans la consigne, on devait changer le Bias et en le changeant, on tombait également à zéro. Donc cela ne change rien, finalement qu'il s'agisse de données random ou numériques choisies celles ci sont peu fiables car elle donne peu de résultat ainsi la seule solution qu'il nous reste ici est le NEAT.

Étape 3) NEAT : neuroévolution de base

Le NEAT est configuré dans un fichier TXT qui va définir tous les paramètres de l'algorithme dans ces paramètres, il y en a des importants à retenir comme:

-Pop_size = Qui définit les individus par génération

-num_inputs = Qui reçoit les valeurs d'entrée

-num_outputs = Qui produit une valeur de sortie (décision de sauter)

-num_hidden = Comme il ne coûte rien de contenir aucune couche cachée au départ, NEAT peut y ajouter des éléments.

Lors du lancement de mon algorithme, j'ai rencontré des petits problèmes.

Probablement de versionning avec "no_fitness_termination" suggérer dans mon erreur .

Une fois cette faute corrigée, j'ai pu voir une évolution des performances:

-Génération 0 = Fitness, Max 199

-Génération 3 = Fitness Max, 12 728, on a vraiment une grosse progression après des générations qui étaient environ de 800 avant celle ci . Mais à la fois on rencontre beaucoup de variations post Génération 3-

Génération 17 = Fitness Max, avec un pic de 19 147, une des plus plus grande variation et pour l'instant la meilleure

-Génération finale = Fitness, Max 10359 . On peut observer ici une légère régression mais celle-ci reste haute comparée à des tests manuels ou random.

En parallèle nous avons doublé la récompense des tuyaux ce qui nous a fait atteindre fitness max de 180 922 , ce qui montre que c'est aussi un paramètre important pour notre jeu.

Étape 4) Amélioration de la fitness et des hyperparamètres

Pour améliorer les paramètres de mon apprentissage j'ai ajusté les paramètres comme le passage de la population à 100 individus, renforcé le bonus tuyau franchi en un facteur de 500 et ou plus, et une augmentation de génération.

Ainsi, on peut voir qu'une population plus vaste permet d'explorer davantage tous les paramètres et devoirs, ce qui conviendrait aux jeux pour l'améliorer et faire en sorte de pouvoir gagner des parties.

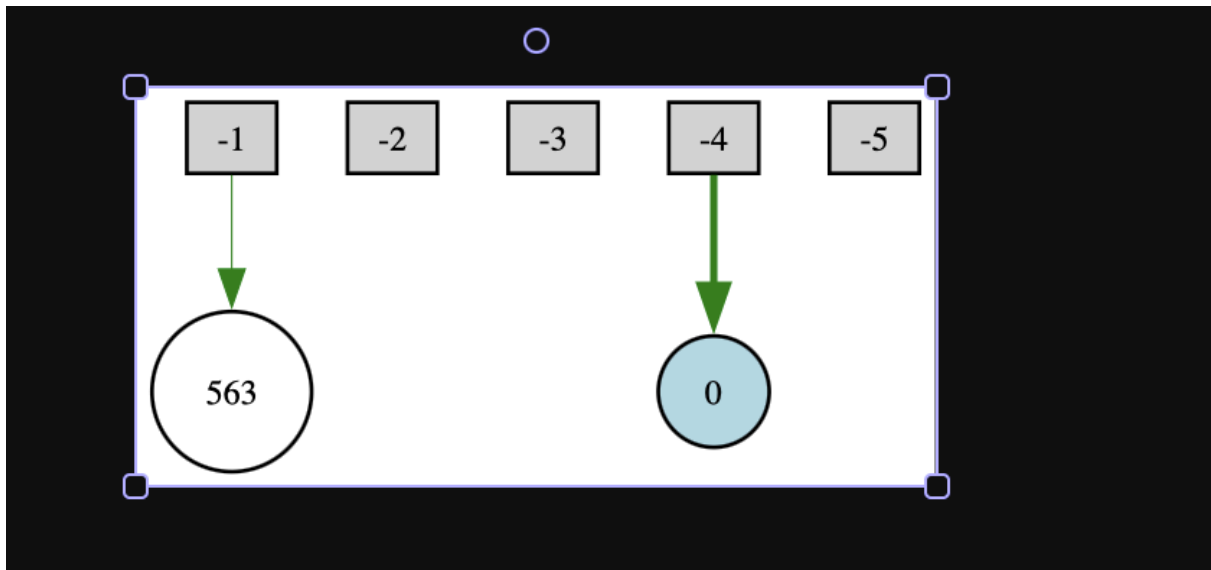
Grâce à ces quelques modifications, la fitness maximale dépasse encore bel et bien les 1000.

Dans cette partie quatre c'est aussi à ce moment-là que les difficultés ont commencé pour moi par exemple pour mettre les 50 générations, je n'avais pas eu l'intuition de les mettre dans N_generation mais je les ai mise dans generation_intervalle et donc grâce à la sortie de mon Terminal j'ai pu voir ce qui allait pas mais c'est vrai que cela m'a prit pas mal de temps.

Ensuite pour la partie [Train.py](#) c'est à ce moment où ça s'est un peu plus corsé, c'est au niveau du get.state_spaces, j'ai pu voir que au niveau de la paramétrage de la generation best_fitness , avg_fitness , ils étaient déjà défini mais il y avait quelques nuances à apporter au final ces nuances changer sans trop de paramètres sensibles , c'est à dire sans trop toucher ce qui avait été corrigé pour éviter de casser à nouveau tout le code juste en ajustant ces détails . Le fitness plot est monté à 31 286. Donc l'algorithme avec NEAT est constamment en amélioration

Étape 5) Visualisation et analyse

Pour illustrer notre analyse nous finissons avec de la visualisation grâce notamment à ia_visualize et play_ia . ia_visualize est un schéma du réseau de neurones avec ces 5 entrées de -1 à -5 ,avec 2 connexions positives à la sortie ,et leurs poids de 563 pour -1(entrée verticale) et 0 pour -4(bord du haut du Gap).



Nous avons ensuite le `play_ia` l'IA qui joue en temps réel j'ai pu obtenir un affichage qui est malheureusement flou , cela peut s'expliquer avec `FlappyBirdEnv` que je n'ai pas su bien paramétrer.

Flappy Bird - IA

Score : █

Frames : ███

Sortie réseau : █.███

Conclusion:

Ce projet à montrer le potentiel de NEAT au détriment des valeurs aléatoires ou numériques choisies qui peine à définir les bons poids NEAT en quelques run parvient à y arriver avec des résultats corrects.